

EA-0051 - AQELYN Engineering Archive

EA-0051 - AQELYN Implementation Blueprint & Coding Readiness Baseline

Publication Date: 2026-07-09

Status: FULL_COMPLETE Publication Source

Section 001 - Document Control

Project: AQELYN

Engineering Archive: EA-0051

Title: AQELYN Implementation Blueprint & Coding Readiness Baseline

Document Type: Master Engineering Archive

Publication Standard: FULL_COMPLETE

Source of Truth: EA-0051_Master.md

Derived Artifacts: PDF, HTML, README, manifest.json, matrices, diagrams, examples, ZIP

Repository Status: Immutable

Classification: Internal Engineering

Date: 2026-07-09

Purpose: This archive closes the design-to-code gap for AQELYN by defining the coding order, runtime stack, repository mapping, interfaces, acceptance gates, implementation milestones, and build/test standards required to begin implementation without changing the established repository structure.

Section 002 - Revision History

Version 1.0 - Initial FULL_COMPLETE publication. Establishes the implementation readiness baseline for coding AQELYN after completion of the pre-coding Engineering Archive series through EA-0050.

Change policy: all later implementation changes shall be captured as Architecture Decision Records, linked to the Engineering Journal, and reflected in requirements/traceability updates before release.

Section 003 - Executive Summary

AQELYN has completed its architecture and planning baseline through EA-0050. The remaining pre-coding work is not additional conceptual design; it is implementation readiness. This archive defines the executable engineering plan needed to start coding safely.

The baseline specifies the first buildable runtime slice, core interfaces, repository file mapping, component order, development standards, testing strategy, and release gates. It preserves the immutable repository structure and translates the engineering archives into a practical coding program.

The recommended implementation starts with a Minimum Viable AQELYN Runtime: Kernel, Universal Object Model, Event Bus, Evidence primitives, Trust/Policy stubs, configuration, logging, and test harness. Higher engines are then

implemented incrementally on top of this stable foundation.

Section 004 - Implementation Readiness Decision

Decision: AQELYN is ready to enter coding after publication of EA-0051.

Rationale: The architectural baseline exists through EA-0050, the repository structure is fixed, the core subsystem dependencies are known, and the first implementation milestones can be executed without further redesign.

Constraint: coding shall not change the immutable top-level repository structure. Any new internal subdirectory must be justified by implementation need and must remain inside the approved top-level folders.

Section 005 - Repository Standard Confirmation

The top-level repository remains fixed:

AQELYN/

- archive/
- blueprint/
- docs/
- src/
- tests/
- tools/
- build/
- releases/
- scripts/
- assets/
- examples/
- plugins/
- sdk/
- api/
- README.md

No implementation milestone may rename, remove, or replace these folders. Generated Engineering Archives remain under archive/ and release artifacts under releases/.

Section 006 - Coding Order Overview

The coding program shall proceed from platform primitives to higher-order engines.

C-001 Foundation Runtime: Kernel, object model, events, configuration, logging, errors, test harness.

C-002 Evidence and Persistence Primitives: evidence objects, hashes, storage adapters, journal records.

C-003 Knowledge Graph Core: nodes, edges, relationship registry, graph adapters.

C-004 Trust and Policy Core: trust score model, deterministic policy evaluation, rule registry.

C-005 Mission and Workflow Core: mission context, workflow dispatcher, remediation task model.

C-006 Identity Baseline: ISPM core, identity inventory, posture scoring.

C-007 Machine Identity Governance: service accounts, workload identities, credential/certificate governance.

C-008 Compliance/Governance Integrations: mappings, control evidence, audit reporting.

C-009 API and SDK Hardening: stable contracts, client SDK, plugin contracts.

C-010 Release Candidate: integration tests, security tests, documentation, package release.

Section 007 - Runtime Stack Baseline

Recommended initial runtime stack:

Language: Python 3.12+

Packaging: pyproject.toml with hatchling or setuptools

Testing: pytest, pytest-cov, hypothesis for property tests

Typing: mypy strict profile for core modules

Formatting: ruff format

Linting: ruff check

Configuration: TOML/YAML loaded into typed configuration objects

Logging: structured JSON logs

Event serialization: JSON canonical form initially, with later optional binary encoding

API: FastAPI-compatible service layer when HTTP boundary is introduced

Persistence: pluggable adapters; local file/sqlite for dev; production adapters later

Cryptography: standard libraries and vetted platform crypto only; no custom crypto

Section 008 - Source Tree Mapping

src/aqelyn/

- kernel/ runtime lifecycle, service registry, health model
- object_model/ universal object model, identifiers, metadata
- event_bus/ event envelope, dispatcher, subscriptions
- evidence/ evidence references, hash chains, evidence store interface
- knowledge_graph/ graph nodes, relationships, graph adapter interface
- trust/ trust score model, trust evaluation contracts
- mission/ mission context and prioritization contracts
- workflow/ workflow definitions and dispatcher
- policy/ policy model, rules, deterministic evaluator
- compliance/ control mapping and compliance evidence
- identity/ human and machine identity primitives
- ispm/ identity posture management implementation
- machine_identity/ non-human identity governance implementation
- api/ internal service interfaces
- sdk/ client-facing helpers
- plugins/ plugin contracts and provider adapters
- common/ errors, time, serialization, validation, logging

tests/ mirrors src/aqelyn with unit, integration, contract, and system tests.

Section 009 - First Buildable Slice

The first buildable slice shall implement enough runtime behavior to execute deterministic tests without external infrastructure.

Required capabilities:

- create AQELYNObject instances
- create AQELYNEvent envelopes
- publish and subscribe events in memory
- register kernel services
- record evidence references with SHA-256 hashes
- evaluate a sample policy rule
- calculate a sample trust score
- execute unit tests and produce coverage
- run a local smoke test from examples/

Section 010 - Core Engineering Interfaces

Initial protocols/classes:

AQELYNService: start(), stop(), health(), name

AQELYNObject: id, type, version, attributes, metadata

AQELYNEvent: id, type, source, timestamp, correlation_id, payload, evidence_refs

EventBus: publish(event), subscribe(event_type, handler)

EvidenceStore: add_evidence(record), get_evidence(id), verify(id)

PolicyEvaluator: evaluate(object, policy_set, context)

TrustEvaluator: evaluate(object, evidence, context)

Plugin: metadata(), capabilities(), initialize(), shutdown()

Connector: discover(), sync(), health()

Section 011 - Kernel Implementation Plan

The AQELYN Kernel is the first coded module. It provides lifecycle control, service registration, health checks, configuration loading, runtime identity, and controlled shutdown.

Kernel responsibilities:

- load configuration
- register services
- initialize services in dependency order
- expose runtime health
- manage graceful shutdown
- provide service lookup
- emit lifecycle events

Kernel exclusions for C-001: clustering, distributed scheduling, persistent service registry, external secrets integration.

Section 012 - Universal Object Model Implementation Plan

The Universal Object Model is implemented before domain engines. It defines canonical identifiers, type names, versioned schemas, metadata, attributes, relationships, and validation.

Minimum object fields:

- object_id
- object_type
- schema_version
- created_at
- updated_at
- source
- labels
- attributes
- relationships
- evidence_refs

The model shall be serialization-stable so identical objects produce deterministic canonical JSON.

Section 013 - Event Bus Implementation Plan

C-001 uses an in-memory event bus to establish event semantics before production transport selection.

Required features:

- event envelope validation
- subscription by event type
- wildcard subscription for tests
- correlation IDs
- publish result metadata
- error isolation between handlers
- event journal option for dev mode

Later milestones may add durable queues, external brokers, ordering policies, replay, and partitioning.

Section 014 - Evidence Engine Stub Plan

Evidence primitives shall be implemented early because every later engine depends on evidence-backed decisions.

C-001 evidence features:

- evidence record model
- SHA-256 digest calculation
- evidence reference object
- verification method

- local file evidence store
- manifest integration

C-002 extends this with chains, retention policy, signing hooks, and immutable journals.

Section 015 - Policy Engine Stub Plan

The first policy implementation shall be deterministic and intentionally small.

C-001 policy capabilities:

- policy document model
- rule identifier
- rule version
- evaluation input
- result: pass/fail/warn/not_applicable
- explanation string
- evidence references

No policy rule may depend on wall-clock time unless the time is explicitly provided as evaluation context.

Section 016 - Trust Engine Stub Plan

The first trust implementation provides a typed TrustScore and deterministic scoring interface.

Trust score fields:

- subject_id
- score 0-100
- confidence 0-100
- contributing_factors
- evidence_refs
- evaluated_at
- evaluator_version

The stub shall support repeatable test vectors and shall not perform behavioral analytics until later milestones.

Section 017 - Plugin and Connector Contract

Provider integrations shall be plugins, not hard-coded core dependencies.

Plugin metadata:

- plugin_id
- name
- version
- provider
- capabilities
- required_permissions

- supported_object_types

Connector operations:

- discover full inventory
- sync delta changes
- validate credentials/configuration
- report health
- emit discovery events

A failed connector must not crash the kernel or halt independent connectors.

Section 018 - API Contract Baseline

Initial API contracts are internal and may be implemented as Python interfaces before HTTP services are introduced.

Stable API domains:

- /objects
- /events
- /evidence
- /policies
- /trust
- /identity
- /machine-identities
- /health

All external API exposure requires versioning and contract tests before release.

Section 019 - Configuration Standard

Configuration shall be environment-aware, typed, validated at startup, and safe by default.

Configuration domains:

- runtime
- logging
- event_bus
- evidence
- policy
- trust
- plugins
- persistence
- api
- security

Secrets shall not be stored directly in static configuration files. Development placeholders are permitted only in examples/ and must be clearly marked non-production.

Section 020 - Logging and Error Standard

All modules shall use structured logging with consistent fields:

- timestamp
- level
- component
- operation
- correlation_id
- object_id when applicable
- event_id when applicable
- outcome
- duration_ms
- error_code when applicable

Errors shall be typed. Core errors: AQELYNError, ConfigurationError, ValidationError, ServiceLifecycleError, EventPublishError, PolicyEvaluationError, EvidenceVerificationError, ConnectorError.

Section 021 - Testing Baseline

Required test layers:

- unit tests for every core object and service
- contract tests for public interfaces
- integration tests for in-memory runtime
- smoke tests under examples/
- regression tests for canonical serialization
- property tests for deterministic scoring where applicable

Minimum gate for C-001: all tests pass, coverage report produced, no critical lint failures.

Section 022 - Security Coding Baseline

Security rules for coding:

- no hard-coded secrets
- no custom cryptography
- validate all external inputs
- deterministic policy and trust evaluation
- safe defaults
- explicit permissions in connector manifests
- signed/hashable publication artifacts
- secure error messages that do not leak secrets
- dependency review before release

Section 023 - Implementation Milestones C-001 to C-010

C-001: Foundation Runtime
C-002: Evidence and Persistence
C-003: Knowledge Graph Core
C-004: Trust and Policy Core
C-005: Mission and Workflow Core
C-006: Identity Posture Core
C-007: Machine Identity Governance Core
C-008: Compliance and Governance Reporting
C-009: API, SDK, Plugins
C-010: Release Candidate and Hardening

Each milestone produces code, tests, examples, updated docs, engineering journal entry, and release notes.

Section 024 - Definition of Done

A coding milestone is done only when:

- implementation exists in approved repository folders
- unit tests pass
- contract tests pass where applicable
- smoke example runs
- documentation is updated
- traceability is updated
- no critical security findings remain
- changelog/release note is written
- generated artifacts are reproducible

Section 025 - Backlog Seed

Initial epics:

1. Project bootstrap and packaging
2. Kernel lifecycle
3. Universal object model
4. Event bus
5. Evidence records
6. Policy evaluator
7. Trust score model
8. Plugin contracts
9. Identity object models
10. Developer documentation
11. Test harness
12. CI workflow definition
13. Example runtime
14. Release packaging

Section 026 - Dependency Graph

Kernel -> Object Model -> Event Bus -> Evidence -> Policy/Trust -> Knowledge Graph -> Mission/Workflow -> Identity Engines -> Compliance -> API/SDK/Plugins -> Release.

No higher-order engine may bypass the object model, event model, or evidence model. All subsystem outputs must be representable as AQELYN objects/events/evidence references.

Section 027 - Build and Release Pipeline

Recommended local commands:

- `python -m venv .venv`
- `pip install -e .[dev]`
- `ruff check src tests`
- `ruff format --check src tests`
- `mypy src`
- `pytest --cov=aqelyn`
- `python examples/smoke_runtime.py`

Release artifacts shall be generated under build/ and copied to releases/ only after validation gates pass.

Section 028 - Documentation to Code Mapping

docs/ holds human-readable engineering guidance.

blueprint/ holds implementation blueprint and milestone maps.

archive/ holds immutable Engineering Archives.

src/ holds product implementation.

tests/ holds verification artifacts.

api/ holds API specifications.

sdk/ holds client SDKs.

plugins/ holds provider extensions.

examples/ holds runnable examples.

tools/ and scripts/ hold build, validation, and publication automation.

Section 029 - Coding Risk Register

Risk: architecture drift. Mitigation: traceability checks and ADRs.

Risk: premature provider coupling. Mitigation: plugin contracts.

Risk: under-tested event semantics. Mitigation: contract tests and deterministic fixtures.

Risk: unclear object model. Mitigation: schema validation and canonical serialization.

Risk: security shortcuts during prototyping. Mitigation: secure coding baseline and CI gates.

Risk: oversized first milestone. Mitigation: strict C-001 scope.

Section 030 - Architecture Decision Records

ADR process:

- create docs/adr/ADR-xxxx.md
- record context, decision, consequences, affected EAs, affected code modules
- link to requirements and traceability entries
- never use ADRs to bypass immutable repository rules

Initial ADRs recommended:

ADR-0001 Python runtime baseline

ADR-0002 In-memory event bus for C-001

ADR-0003 Canonical JSON serialization

ADR-0004 Plugin-first provider integration

ADR-0005 Evidence hash baseline

Section 031 - Requirements Matrix

REQ-0051-001: Define coding order. Verification: review milestone plan.

REQ-0051-002: Define runtime stack. Verification: review build baseline.

REQ-0051-003: Define repository mapping. Verification: compare against immutable structure.

REQ-0051-004: Define core interfaces. Verification: contract checklist.

REQ-0051-005: Define first buildable slice. Verification: C-001 acceptance criteria.

REQ-0051-006: Define testing gates. Verification: test strategy review.

REQ-0051-007: Define security coding baseline. Verification: security review.

REQ-0051-008: Define release pipeline. Verification: build command review.

REQ-0051-009: Define ADR process. Verification: governance review.

REQ-0051-010: Define readiness approval. Verification: final engineering review.

Section 032 - Traceability Matrix

REQ-0051-001 -> Coding Order Overview -> C-001..C-010 -> Milestone review -> EV-0051-001

REQ-0051-002 -> Runtime Stack Baseline -> pyproject/tooling -> Build validation -> EV-0051-002

REQ-0051-003 -> Source Tree Mapping -> repository skeleton -> Structure validation -> EV-0051-003

REQ-0051-004 -> Core Interfaces -> protocol definitions -> Contract tests -> EV-0051-004

REQ-0051-005 -> First Buildable Slice -> foundation runtime -> Smoke test -> EV-0051-005

REQ-0051-006 -> Testing Baseline -> pytest/coverage -> Test report -> EV-0051-006

REQ-0051-007 -> Security Coding Baseline -> secure coding gates -> Security review -> EV-0051-007

REQ-0051-008 -> Build Pipeline -> CI/local commands -> Build log -> EV-0051-008

REQ-0051-009 -> ADR Process -> docs/adr -> ADR audit -> EV-0051-009

REQ-0051-010 -> Readiness Approval -> engineering review -> Approval record -> EV-0051-010

Section 033 - Engineering Journal

EA-0051 records the transition from design/planning into coding. The project has enough architecture to start implementation. The highest engineering value now comes from building the foundation runtime while keeping the full

archive traceability intact.

Key journal entry: do not attempt to implement all engines simultaneously. Build the kernel and primitives first, then layer engines in dependency order.

Section 034 - Example Artifact: pyproject.toml

```
[project]
name = "project-aqelyn"
version = "0.1.0"
description = "AQELYN Cyber Security Operating Environment"
requires-python = ">=3.12"
dependencies = []

[project.optional-dependencies]
dev = ["pytest", "pytest-cov", "ruff", "mypy", "hypothesis"]

[tool.ruff]
line-length = 100

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Section 035 - Example Artifact: smoke_runtime.py

```
from aqelyn.kernel import AQELYNKernel
from aqelyn.event_bus import InMemoryEventBus, AQELYNEvent

kernel = AQELYNKernel()
event_bus = InMemoryEventBus()
kernel.register(event_bus)
kernel.start()

event_bus.publish(AQELYNEvent(type="AQELYN.RuntimeStarted", payload={"status": "ok"}))

assert kernel.health().status == "healthy"
kernel.stop()
```

Section 036 - Example Artifact: Core Interface Sketch

```
class AQELYNService:
    def start(self) -> None: ...
    def stop(self) -> None: ...
    def health(self) -> HealthStatus: ...

class EventBus:
    def publish(self, event: AQELYNEvent) -> PublishResult: ...
```

```
def subscribe(self, event_type: str, handler: EventHandler) -> Subscription: ...
```

```
class EvidenceStore:
```

```
def add(self, record: EvidenceRecord) -> EvidenceRef: ...
```

```
def verify(self, ref: EvidenceRef) -> bool: ...
```

Section 037 - Publication Manifest Description

This FULL_COMPLETE package contains the approved master Markdown, PDF, HTML, README, manifest.json, requirements matrix, traceability matrix, engineering journal, example artifacts, and five SVG diagrams.

All generated artifacts derive from the approved EA-0051 master source. manifest.json contains SHA-256 hashes for integrity verification.

Section 038 - Final Engineering Review

Review result: APPROVED FOR CODING.

The remaining pre-coding work has been captured in this implementation readiness baseline. Coding may begin with C-001 Foundation Runtime.

Conditions:

- repository structure remains immutable
- C-001 scope remains small
- every module includes tests
- every architecture deviation requires ADR
- every release artifact is reproducible

Section 039 - Coding Start Authorization

Authorized first coding milestone: C-001 Foundation Runtime.

First files to create:

- pyproject.toml
- src/aqelyn/__init__.py
- src/aqelyn/kernel/__init__.py
- src/aqelyn/kernel/service.py
- src/aqelyn/object_model/base.py
- src/aqelyn/event_bus/events.py
- src/aqelyn/event_bus/in_memory.py
- src/aqelyn/evidence/model.py
- src/aqelyn/policy/model.py
- src/aqelyn/trust/model.py
- tests/test_kernel.py
- tests/test_event_bus.py
- examples/smoke_runtime.py

Section 040 - Publication Readiness

EA-0051 is publication ready when the package includes all required FULL_COMPLETE artifacts and passes render/hash verification.

After EA-0051 publication, the next action is coding Milestone C-001, not further design expansion.

Section 041 - Implementation Appendix 1

This appendix expands implementation readiness guidance for module group 1.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 042 - Implementation Appendix 2

This appendix expands implementation readiness guidance for module group 2.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 043 - Implementation Appendix 3

This appendix expands implementation readiness guidance for module group 3.

Coding guidance:

- keep module boundaries explicit

- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 044 - Implementation Appendix 4

This appendix expands implementation readiness guidance for module group 4.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 045 - Implementation Appendix 5

This appendix expands implementation readiness guidance for module group 5.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 046 - Implementation Appendix 6

This appendix expands implementation readiness guidance for module group 6.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 047 - Implementation Appendix 7

This appendix expands implementation readiness guidance for module group 7.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 048 - Implementation Appendix 8

This appendix expands implementation readiness guidance for module group 8.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 049 - Implementation Appendix 9

This appendix expands implementation readiness guidance for module group 9.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 050 - Implementation Appendix 10

This appendix expands implementation readiness guidance for module group 10.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 051 - Implementation Appendix 11

This appendix expands implementation readiness guidance for module group 11.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters

- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 052 - Implementation Appendix 12

This appendix expands implementation readiness guidance for module group 12.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 053 - Implementation Appendix 13

This appendix expands implementation readiness guidance for module group 13.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 054 - Implementation Appendix 14

This appendix expands implementation readiness guidance for module group 14.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 055 - Implementation Appendix 15

This appendix expands implementation readiness guidance for module group 15.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 056 - Implementation Appendix 16

This appendix expands implementation readiness guidance for module group 16.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 057 - Implementation Appendix 17

This appendix expands implementation readiness guidance for module group 17.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 058 - Implementation Appendix 18

This appendix expands implementation readiness guidance for module group 18.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 059 - Implementation Appendix 19

This appendix expands implementation readiness guidance for module group 19.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references

- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.

Section 060 - Implementation Appendix 20

This appendix expands implementation readiness guidance for module group 20.

Coding guidance:

- keep module boundaries explicit
- write tests before broad integration
- prefer deterministic pure functions for rules and scoring
- isolate I/O in adapters
- keep provider-specific logic in plugins
- represent state transitions as events
- link significant runtime decisions to evidence references
- preserve stable identifiers across logs, events, and objects

Acceptance for this appendix: the guidance is reflected in module README files, tests, and engineering journal updates during the relevant coding milestone.